

**Travail de Fin d'Études**

# **Snow Wonder Festival**

*Application web événementielle single-page pour un festival hivernal de sculptures de neige*

**Claire Siboni**

Promoteur : Guillaume Duribreux

IFAPME Wallonie Picarde — Développeur Web Front-End X75

Année académique 2025–2026





**Travail de Fin d'Études**

# **Snow Wonder Festival**

*Application web événementielle single-page pour un festival hivernal de sculptures de neige*

**Claire Siboni**

Promoteur : Guillaume Duribreux

IFAPME Wallonie Picarde — Développeur Web Front-End X75

Année académique 2025–2026



## Remerciements

Je souhaite adresser mes premiers remerciements à mon promoteur, Monsieur Guillaume Duribreux, pour sa disponibilité, ses retours constructifs et la confiance qu'il m'a accordée tout au long de ce projet.

Je remercie également Martial Parfait et François Racquez pour la qualité de leurs enseignements et pour avoir su transmettre leur passion du développement web avec pédagogie et bienveillance.

Une pensée particulière va à mes camarades de formation Viktor, Guillaume et Killian, derniers survivants après deux longues années.

Une mention spéciale à mes sœurs, Corentine et Annelise, testeuses officielles de tous mes projets, qui ont su m'aiguiller par leurs précieux conseils.

Enfin, je remercie du fond du cœur ma famille et mes proches, dont le soutien quotidien a été une source d'énergie précieuse tout au long de cette aventure.

## Table des matières

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>Introduction.....</b>                                            | <b>1</b>  |
| Contexte et objectifs.....                                          | 1         |
| Méthodes et technologies développées.....                           | 1         |
| <b>Chapitre 1 — Analyse des besoins et planification.....</b>       | <b>2</b>  |
| 1.1 Analyse des besoins (Impact Mapping).....                       | 2         |
| 1.2 User Stories.....                                               | 2         |
| Périmètre visiteur (front-office public).....                       | 2         |
| Périmètre participant au concours.....                              | 3         |
| Périmètre visiteur souhaitant assister au festival.....             | 3         |
| Périmètre administrateur (back-office).....                         | 3         |
| Périmètre agent de contrôle (scanner).....                          | 3         |
| 1.3 Planning d'exécution.....                                       | 3         |
| <b>Chapitre 2 — Conception UI/UX et charte graphique.....</b>       | <b>5</b>  |
| 2.1 Identité visuelle et charte graphique.....                      | 5         |
| 2.2 Inspirations design.....                                        | 5         |
| 2.3 Wireframes et conception de l'interface.....                    | 5         |
| <b>Chapitre 3 — Architecture technique et choix des outils.....</b> | <b>7</b>  |
| 3.1 React et Vite : l'architecture composant.....                   | 7         |
| 3.2 TailwindCSS v4 : le design utilitaire.....                      | 7         |
| 3.3 Supabase : le Backend-as-a-Service.....                         | 7         |
| 3.4 Leaflet.js et Mapbox : la carte interactive.....                | 7         |
| <b>Chapitre 4 — Analyse de la base de données.....</b>              | <b>9</b>  |
| 4.1 Structure des tables.....                                       | 9         |
| 4.2 Relations entre les tables.....                                 | 9         |
| <b>Chapitre 5 — Schéma du chemin des données.....</b>               | <b>10</b> |
| 5.1 Vue d'ensemble de l'architecture.....                           | 10        |
| 5.2 Parcours 1 — Chargement dynamique des éditions.....             | 10        |
| 5.3 Parcours 2 — Inscription au concours.....                       | 10        |
| 5.4 Parcours 3 — Réservation de billets.....                        | 10        |
| 5.5 Parcours 4 — Formulaire de contact.....                         | 10        |
| 5.6 Parcours 5 — Galerie photo (lecture publique).....              | 10        |
| 5.7 Parcours 6 — Scan à l'entrée (TicketScanner).....               | 10        |
| 5.8 Parcours 7 — Authentification Back-Office.....                  | 11        |
| 5.9 Parcours 8 — Back-Office (opérations CRUD admin).....           | 11        |
| 5.10 Tableau récapitulatif des permissions RLS.....                 | 11        |
| 5.11 Points clés de sécurité.....                                   | 11        |
| <b>Chapitre 6 — Fonctionnalités clés du site.....</b>               | <b>13</b> |
| 6.1 Le compte à rebours dynamique (Hero).....                       | 13        |
| 6.2 Gestion dynamique des éditions.....                             | 13        |

|                                                               |           |
|---------------------------------------------------------------|-----------|
| 6.3 Les formulaires : inscription et billetterie.....         | 13        |
| 6.4 La galerie photo dynamique.....                           | 14        |
| 6.5 Le système de scan à l'entrée (TicketScanner).....        | 14        |
| 6.6 Le panneau d'administration (Back-Office).....            | 14        |
| 6.7 Sécurité et protection des données.....                   | 15        |
| <b>Chapitre 7 — Déploiement et mise en production.....</b>    | <b>16</b> |
| 7.1 Déploiement sur Vercel.....                               | 16        |
| 7.2 Supabase Edge Functions.....                              | 16        |
| 7.3 Optimisation des performances.....                        | 16        |
| Découpage du bundle JavaScript (code splitting).....          | 16        |
| Lazy loading et priorisation des images.....                  | 16        |
| Conversion WebP et réduction du poids des images.....         | 16        |
| <b>Chapitre 8 — RGPD, cookies et pollution numérique.....</b> | <b>17</b> |
| 8.1 RGPD et protection des données personnelles.....          | 17        |
| 8.2 Gestion des cookies.....                                  | 17        |
| 8.3 Pollution numérique.....                                  | 17        |
| <b>Chapitre 9 — Estimation budgétaire.....</b>                | <b>18</b> |
| 9.1 Hypothèses de calcul.....                                 | 18        |
| 9.2 Phases de développement.....                              | 18        |
| 9.3 Frais annexes (première année).....                       | 18        |
| 9.4 Maintenance annuelle.....                                 | 18        |
| 9.5 Coût total première année.....                            | 19        |
| <b>Conclusion.....</b>                                        | <b>20</b> |
| Intégration des savoirs et savoir-faire.....                  | 20        |
| Adéquation entre objectifs et résultats.....                  | 20        |
| Difficultés rencontrées et solutions apportées.....           | 20        |
| Réflexion personnelle.....                                    | 21        |
| <b>Bibliographie.....</b>                                     | <b>22</b> |
| Ouvrages et règlements.....                                   | 22        |
| Documentations techniques.....                                | 22        |
| Sources économiques.....                                      | 22        |
| Crédits photographiques.....                                  | 22        |
| Intelligence artificielle.....                                | 23        |
| <b>Table des figures.....</b>                                 | <b>24</b> |

## Introduction

Dans le cadre de ma formation de Développeur Web Front-End à l'IFAPME, j'ai eu l'opportunité de concevoir et de réaliser un projet complet de bout en bout, simulant une collaboration professionnelle réelle entre un client et un développeur. Ce Travail de Fin d'Études constitue l'aboutissement de deux années de formation et vise à démontrer la maîtrise des compétences acquises : analyse des besoins, planification, conception UI/UX, développement, intégration d'API et mise en ligne.

Le projet que j'ai choisi de réaliser est Snow Wonder Festival, un site web événementiel de type single-page application (SPA) dédié à un festival hivernal de sculptures de neige organisé en trois éditions — en décembre 2026, janvier 2027 et février 2027 — sur le territoire belge. Ce thème m'a permis de combiner une identité visuelle forte, des fonctionnalités interactives riches et une architecture technique moderne, tout en répondant précisément aux exigences du cahier des charges.

## Contexte et objectifs

Le brief client exprimait la volonté de créer une expérience en ligne mémorable et engageante, capable de promouvoir les éditions du festival, d'accueillir les inscriptions des participants au concours de sculptures et de gérer la billetterie en ligne. Quatre exigences fondamentales ont guidé l'ensemble du projet :

- Réaliser une application single-page dynamique possédant un front-office public et un back-office d'administration sécurisé ;
- Intégrer une API cartographique (OpenStreetMap via Leaflet.js, enrichie de tuiles personnalisées Mapbox) ;
- Mettre en place un système de réservation en ligne avec confirmation par e-mail automatisée ;
- Garantir la sécurité des données des utilisateurs et l'intégrité du site.

## Méthodes et technologies développées

Pour répondre à ces objectifs, j'ai adopté une approche centrée sur les technologies modernes du développement web front-end :

- React 19 comme bibliothèque d'interface utilisateur, permettant une gestion réactive des composants et une architecture modulaire ;
- Vite comme outil de build, choisi pour sa rapidité de compilation et son expérience développeur optimale ;
- TailwindCSS v4 pour la mise en forme utilitaire et le design responsive ;
- Supabase comme Backend-as-a-Service (BaaS), fournissant la base de données PostgreSQL, l'authentification et les fonctions serverless (Edge Functions en Deno/TypeScript) ;
- Leaflet.js couplé à l'API Mapbox pour la carte interactive ;
- Vercel pour le déploiement continu et l'hébergement.

Ce document présente successivement l'analyse menée en amont du projet, les choix de conception et de technologies, la description détaillée de la mise en œuvre, puis une évaluation personnelle du travail accompli.

## Chapitre 1 — Analyse des besoins et planification

### 1.1 Analyse des besoins (Impact Mapping)

Avant d'écrire la moindre ligne de code, j'ai procédé à une analyse rigoureuse des besoins à partir du brief client. Cette analyse s'est articulée autour de quatre catégories d'acteurs (ou impacts) :

- Les visiteurs : ils ont besoin d'informations claires sur le festival (programme, dates, lieu), d'un accès simple à la billetterie et d'une expérience visuelle immersive qui donne envie de venir.
- Les participants au concours : ils doivent pouvoir s'inscrire facilement à la compétition (catégorie amateur ou professionnelle), choisir leur(s) édition(s) et recevoir une confirmation.
- L'organisateur (l'administrateur) : il a besoin d'un back-office complet pour consulter les inscriptions et les commandes de billets, gérer le contenu du site (textes, galerie photo, gagnants, éditions), sans dépendre d'un développeur pour chaque modification.
- Le système : il doit garantir la sécurité des données, l'envoi automatique des e-mails de confirmation et la disponibilité du site.

De cette analyse sont ressortis les besoins fonctionnels suivants :

| Fonctionnalité                                       | Priorité |
|------------------------------------------------------|----------|
| Présentation du festival (hero, programme, galerie)  | Haute    |
| Carte interactive géolocalisée                       | Haute    |
| Formulaire d'inscription au concours                 | Haute    |
| Réservation de billets avec confirmation par e-mail  | Haute    |
| Interface d'administration sécurisée                 | Haute    |
| Gestion dynamique des éditions depuis le back-office | Haute    |
| Système de scan QR à l'entrée du festival            | Haute    |
| Gestion de la galerie photo depuis l'admin           | Moyenne  |
| Section palmarès / gagnants                          | Moyenne  |
| Newsletter                                           | Basse    |

### 1.2 User Stories

L'analyse des besoins s'est structurée autour de quatre périmètres distincts correspondant aux acteurs identifiés dans l'impact mapping.

#### Périmètre visiteur (front-office public)

- Consulter le programme des trois éditions du festival à venir (décembre 2026, janvier 2027, février 2027)
- Naviguer entre les sections via le menu fixe avec ancrs (#schedule, #gallery, #market, #forms, #map, #winners)
- Visualiser la galerie photo avec lightbox



- Localiser le site du festival et ses zones sur la carte interactive
- Consulter le palmarès des éditions passées
- Accéder au formulaire de contact via le pied de page

**Périmètre participant au concours**

- S'inscrire au concours de sculptures de neige en choisissant sa catégorie (amateur ou professionnel) et son/ses édition(s)
- Être informé en temps réel si son adresse e-mail est déjà inscrite à une édition
- Recevoir un e-mail de confirmation HTML personnalisé après soumission

**Périmètre visiteur souhaitant assister au festival**

- Réserver jusqu'à 10 billets gratuits pour l'édition de son choix
- Fournir ses coordonnées complètes (nom, e-mail, téléphone, adresse postale)
- Opter pour l'abonnement à la newsletter
- Recevoir ses billets PDF individuels par e-mail, chacun portant un QR code unique au format SWF-XXX-XXXXXX

**Périmètre administrateur (back-office)**

- Se connecter de manière sécurisée via e-mail et mot de passe avec double vérification (Supabase Auth + table admin\_users)
- Réinitialiser son mot de passe via un lien sécurisé reçu par e-mail
- Consulter le tableau de bord avec les statistiques clés, les actions rapides et le journal d'activité récente
- Créer, modifier, activer/désactiver et supprimer des éditions sans redéploiement
- Filtrer, consulter et supprimer des inscriptions au concours
- Gérer les commandes de billets et les billets individuels générés
- Ajouter, supprimer, réorganiser et activer/désactiver les images de la galerie
- Modifier les textes éditoriaux du site sans redéploiement
- Saisir et mettre à jour le palmarès des éditions

**Périmètre agent de contrôle (scanner)**

- Accéder à une interface épurée limitée au seul module de scan QR code
- Scanner un billet ou une inscription pour en valider l'entrée en temps réel
- Recevoir un retour visuel immédiat (vert / orange / rouge) selon le statut du document scanné

## 1.3 Planning d'exécution

J'ai structuré le projet en quatre grandes phases :

- Phase 1 — Conception et premiers développements (décembre – janvier) : élaboration de la charte graphique, création du moodboard, réalisation des wireframes, conception du schéma de base de données. Une première version du site a été développée en HTML vanilla, CSS et JavaScript natif, posant les bases de l'architecture et de l'identité visuelle.
- Phase 2 — Refonte technologique et développement front-end (février – avril) : fort de mon expérience en stage et souhaitant adopter des technologies mieux adaptées à une SPA, j'ai migré le projet vers React et

TailwindCSS. Cette phase a également inclus la configuration de Supabase (tables, politiques RLS, authentification) et le développement des composants principaux de la page publique, des formulaires et de l'intégration de la carte Leaflet.

- Phase 3 — Back-end, déploiement et mise en ligne (fin avril – début mai) : développement du panneau d'administration, écriture des Edge Functions (envoi d'e-mails, génération de PDF), module de gestion des éditions, système de scan à l'entrée. Le site a été déployé sur Vercel et le domaine snow-wonder.be configuré.
- Phase 4 — Tests, optimisation et finitions (mai) : tests fonctionnels multi-navigateurs, optimisation des performances (audit Lighthouse, code splitting, conversion WebP), vérification de la sécurité, ajustements visuels et ajout des dernières fonctionnalités.

## Chapitre 2 — Conception UI/UX et charte graphique

### 2.1 Identité visuelle et charte graphique

L'identité visuelle de Snow Wonder Festival repose sur un univers hivernal élégant, à la fois féérique et moderne. Le choix du thème de l'hiver m'a permis de construire une palette chromatique cohérente et évocatrice :

- deep-navy (#0a1628) : bleu nuit profond, utilisé comme couleur de fond principale. Il évoque le ciel d'hiver nocturne et confère au site une ambiance premium.
- midblue (#1e3a5f) : bleu intermédiaire, utilisé dans les dégradés et les éléments de navigation.
- ice-blue (#cae9ff) : bleu glace clair, utilisé pour les textes secondaires et les éléments décoratifs.
- festival-yellow (#e8a94e) : jaune doré chaleureux, couleur d'accentuation principale.

Le fond de la page est un dégradé vertical allant de ice-blue à midblue puis deep-navy, créant une profondeur atmosphérique naturelle. Typographie : DM Serif Display pour les titres, Nunito pour le corps de texte, Rubik pour l'administration.

### 2.2 Inspirations design

La conception visuelle s'est nourrie d'une veille graphique approfondie. Plusieurs sites ont servi de références : And Apartments pour l'élégance des typographies, Move to Be More pour la fluidité du scroll, Tokyo Design Forum pour l'équilibre entre densité et respiration visuelle, Two Hyde Design pour les effets de superposition, Brewitty pour la cohérence de l'identité, Open Lines pour les micro-interactions sur fond sombre, Benjamin Wilkes pour les animations au défilement, Frank Firewood pour l'atmosphère hivernale, et Shirtique Krazykaps pour la mise en valeur photographique.

### 2.3 Wireframes et conception de l'interface

Les wireframes ont été conçus en respectant les principes fondamentaux de l'UX : hiérarchie visuelle claire, parcours utilisateur intuitif et appels à l'action bien positionnés. La page se décompose en sections empilées verticalement : navigation fixe, Hero avec compte à rebours, Programme, Galerie, Marché, Formulaire, Carte, Gagnants et Pied de page (voir Figure 1).



Figure 1 — Page d'accueil — section Hero avec compte à rebours (desktop)

Conception responsive : trois breakpoints ont été définis. Mobile (< 768px) : navigation en menu latéral, formulaires empilés verticalement avec des formes en quart de cercle, galerie en grille 2 colonnes. Tablette (768px–1280px) : formulaires côte à côte dans des cards glassmorphiques. Desktop (> 1280px) : formulaires imbriqués dans deux grands cercles semi-transparentes qui se chevauchent (voir Figures 2 et 3).

**Inscription Artiste**  
Amateur ou professionnel

TYPE  
☐ Amateur ☐ Professionnel

ÉDITION(S) \*

Édition d'Ouverture 6 décembre 2025  
Édition Mi-Hiver 10 janvier 2027  
Grande Finale 7 février 2027

PRÉNOM \* Marie  
NOM \* Dupont

E-MAIL \* marie@example.com

TÉLÉPHONE \* +32 470 00 00 00

☐ J'accepte les conditions générales \*

Réinitialiser S'INSCRIRE

**Vos Billets**  
Gratuit - max 10 billets par personne  
Pour plus de 10 billets, contactez nous à [info@snow-wonder.be](mailto:info@snow-wonder.be)

ÉDITION \*

Édition d'Ouverture 6 décembre 2025  
Édition Mi-Hiver 10 janvier 2027  
Grande Finale 7 février 2027

NOMBRE DE BILLETS \* 1

PRÉNOM \* Marie  
NOM \* Dupont

E-MAIL \* marie@example.com

TÉLÉPHONE \* +32 470 00 00 00

ADRESSE \* Rue du Marché 12

VILLE \* Bruxelles CP \* 1000 PAYS \* Belgique

☐ S'abonner à la newsletter  
☐ J'accepte les conditions générales et la politique de confidentialité \*

Réinitialiser OBTENIR 1 BILLET

Figure 2 — Section formulaires — mise en page desktop (cercles superposés)

[ Insérer capture d'écran ici ]

Figure 3 — Section formulaires — mise en page mobile (quart de cercle)

## Chapitre 3 — Architecture technique et choix des outils

### 3.1 React et Vite : l'architecture composant

Le choix de React comme bibliothèque d'interface s'est imposé naturellement. React permet de découper l'interface en composants réutilisables et indépendants. Chaque section du site correspond à un composant autonome (Hero, ScheduleSection, GallerySection, MarketSection, FormsSection, MapSection, WinnersSection), orchestré par le composant racine App.jsx. L'architecture est organisée en sous-dossiers : buildingBlocks/, modals/ et sections/.

Vite a été préféré à Create React App pour : son démarrage quasi instantané grâce aux modules ES natifs, son Hot Module Replacement ultra-rapide et son build de production optimisé avec tree-shaking et code-splitting automatiques.

### 3.2 TailwindCSS v4 : le design utilitaire

TailwindCSS permet de styliser les composants directement dans le JSX via des classes prédéfinies. Des couleurs personnalisées ont été définies dans la configuration (bg-deep-navy, text-festival-yellow, etc.) pour garantir la cohérence chromatique. Pour les cas plus complexes (animation de ScheduleSection, effets glassmorphiques de FormsSection), des styles CSS dédiés ont été utilisés en complément.

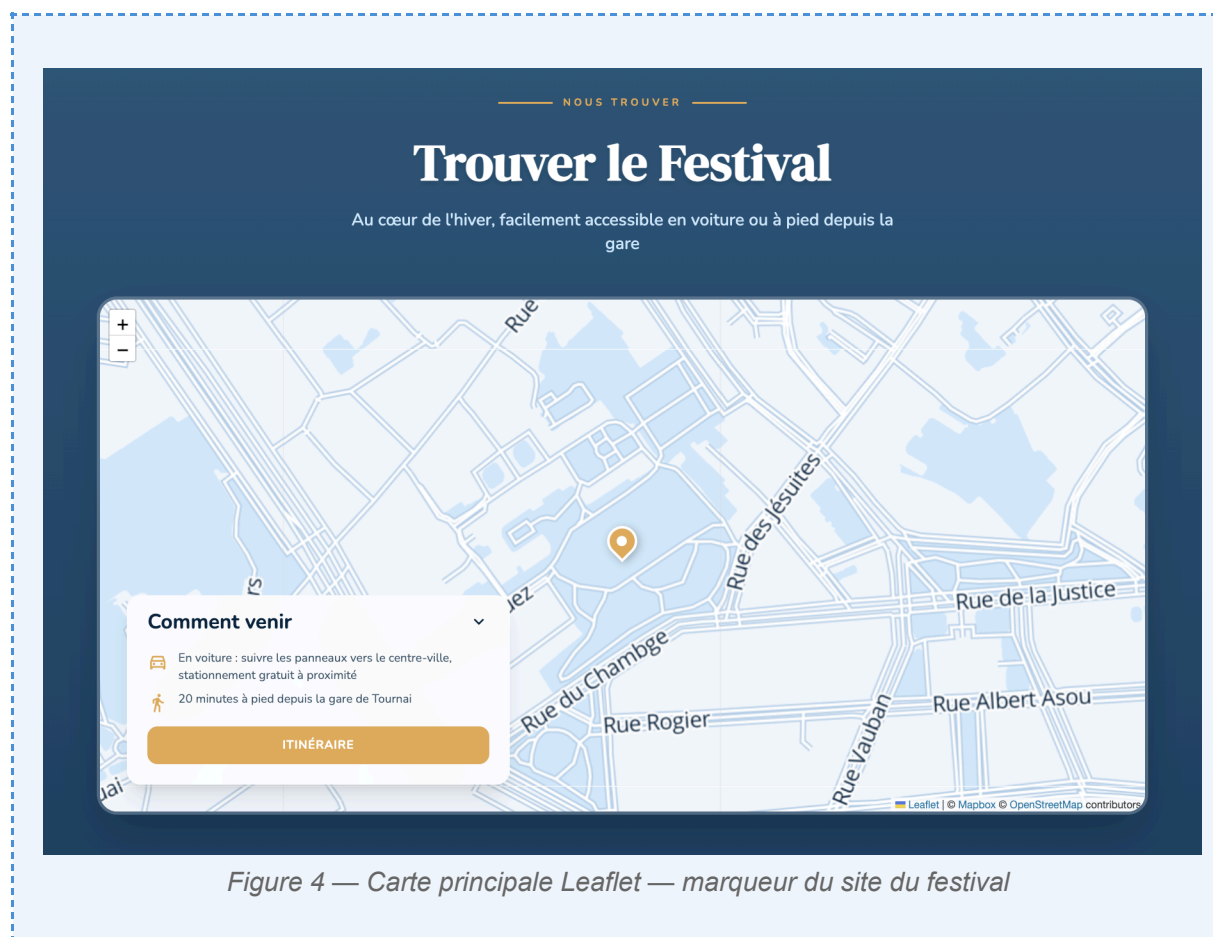
### 3.3 Supabase : le Backend-as-a-Service

Supabase fournit une base de données PostgreSQL, une API REST auto-générée, un système d'authentification et des fonctions serverless. Les avantages clés par rapport à un back-end PHP personnalisé : rapidité de mise en place, Row Level Security granulaire, authentification JWT clé en main et Edge Functions Deno/TypeScript pour les opérations sensibles. La clé publique anonyme (VITE\_SUPABASE\_ANON\_KEY) ne permet que les opérations autorisées par les politiques RLS.

### 3.4 Leaflet.js et Mapbox : la carte interactive

Leaflet.js a été choisi plutôt que Google Maps pour sa gratuité, sa compatibilité avec OpenStreetMap et ses possibilités de personnalisation via des tuiles Mapbox. Le site intègre deux instances Leaflet distinctes.

La première est une carte principale affichant un unique marqueur positionné aux coordonnées GPS précises du site du festival, avec une icône personnalisée en L.divIcon ouvrant un popup avec lien Google Maps Directions (voir Figure 4).



La seconde est une carte du marché, accessible via une modal dédiée, qui affiche neuf marqueurs : huit stands gastronomiques et un marqueur central identifiant le gazebo (voir Figure 5). Les deux cartes sont montées via `useRef` et `useEffect` et nettoyées au démontage pour éviter les fuites mémoire.

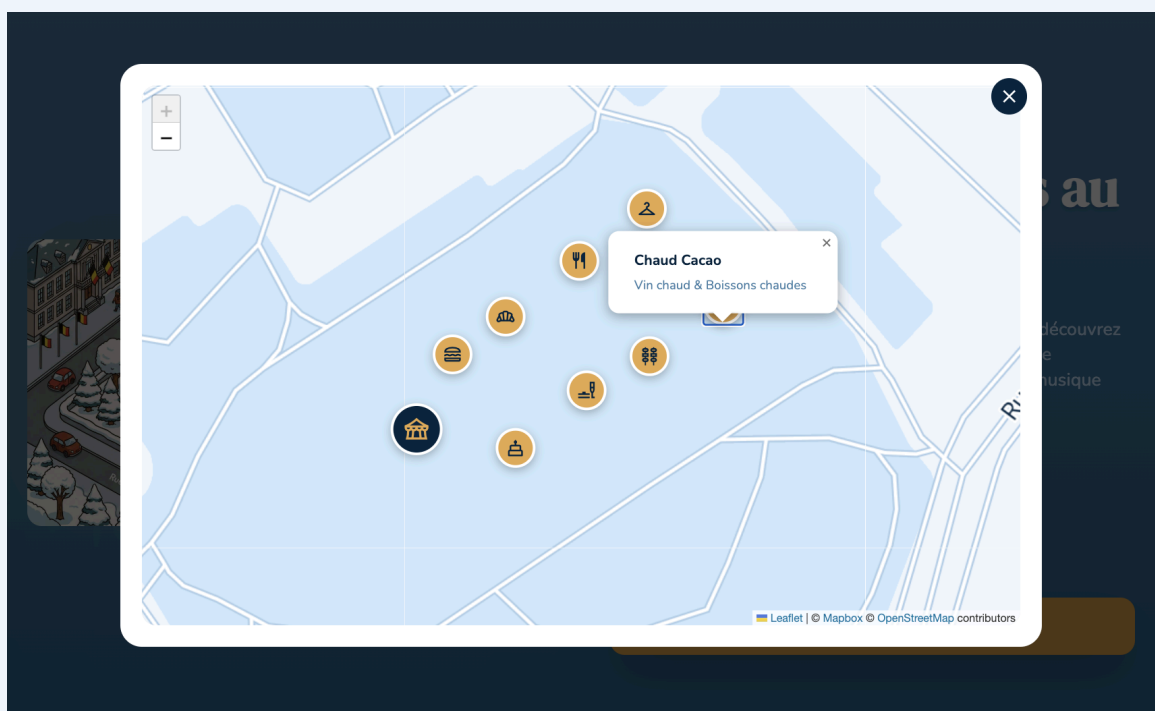


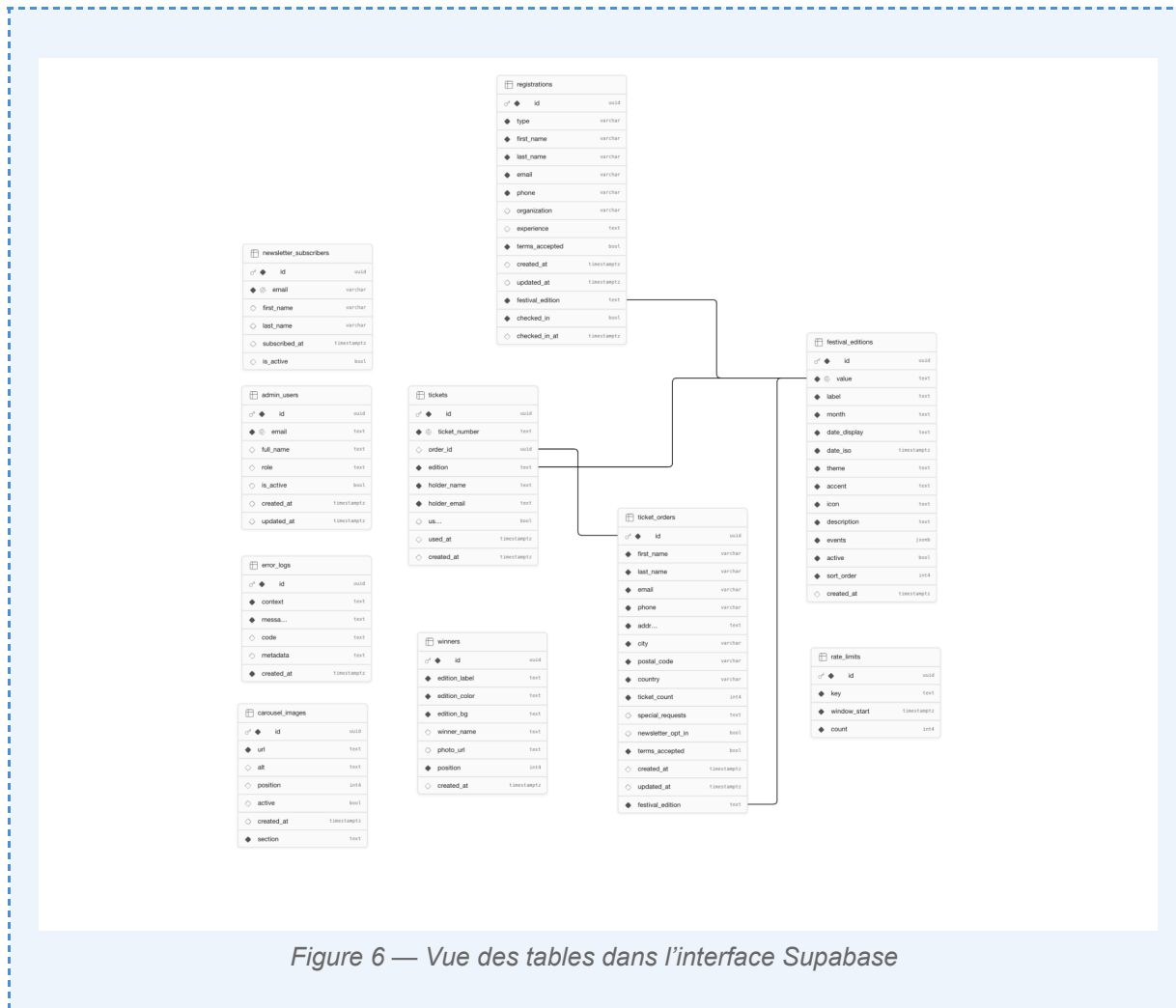
Figure 5 — Carte du marché — modal avec les 9 marqueurs gastronomiques



## Chapitre 4 — Analyse de la base de données

### 4.1 Structure des tables

La base de données PostgreSQL hébergée sur Supabase est composée de dix tables dont voici la description et les relations (voir Figure 6).



La table `registrations` stocke les inscriptions au concours. Elle contient les informations personnelles du participant (prénom, nom, e-mail, téléphone, organisation éventuelle), sa catégorie (amateur ou pro via une contrainte CHECK), l'édition choisie via une clé étrangère vers `festival_editions`, le champ d'expérience et l'acceptation des conditions. Les champs `checked_in` et `checked_in_at` permettent de valider la présence du participant à l'entrée.

La table `ticket_orders` centralise les commandes de billets. Elle enregistre les coordonnées complètes du visiteur, le nombre de billets commandés (limité entre 1 et 10 par contrainte SQL CHECK), l'édition concernée et l'opt-in newsletter.

La table `tickets` représente les billets individuels générés. Chaque billet porte un numéro unique au format SWF-XXX-XXXXXX, est lié à `ticket_orders` (via `order_id`) et à `festival_editions` (via `edition`), et peut être marqué comme utilisé via les champs `used` et `used_at`.

La table `festival_editions` gère les éditions de façon entièrement dynamique, avec ses métadonnées complètes (identifiant `value`, libellé, date ISO, thème, couleur, icône Material Symbols, liste d'événements en JSONB). Les champs `sort_order` et `active` permettent de contrôler l'ordre et la visibilité sans suppression.

La table `admin_users` gère les comptes d'administration indépendamment de `auth.users`, avec un rôle (`admin`, `super_admin` ou `scanner` via contrainte CHECK) et un indicateur `is_active`. Les tables `carousel_images`, `newsletter_subscribers`, `winners`, `rate_limits` et `error_logs` complètent le schéma.

## 4.2 Relations entre les tables

La relation principale est celle entre `ticket_orders` et `tickets` (one-to-many). `registrations` et `ticket_orders` entretiennent chacune une relation many-to-one avec `festival_editions` via une clé étrangère sur le champ `value`. Les composants `Registration.jsx`, `Tickets.jsx` et `ScheduleSection.jsx` consomment les éditions via le hook `useEditions`. Les tables `admin_users`, `carousel_images`, `winners`, `rate_limits` et `error_logs` sont indépendantes et communiquent via les politiques RLS.

## Chapitre 5 — Schéma du chemin des données

Ce chapitre décrit l'intégralité des flux de données de l'application, depuis le navigateur du visiteur jusqu'aux services tiers, en passant par Supabase et les Edge Functions.

### 5.1 Vue d'ensemble de l'architecture

L'application s'articule autour de trois pôles : le navigateur client (React 19 + Vite, hébergé sur Vercel), Supabase (PostgreSQL avec RLS, Auth JWT, Edge Functions Deno/TypeScript et Storage), et les services tiers (api.mapbox.com pour les tuiles, api.qrserver.com pour les QR codes, Vercel Analytics et Google Maps). Les envois d'e-mails transitent par un serveur SMTP via Nodemailer, appelé depuis les Edge Functions.

### 5.2 Parcours 1 — Chargement dynamique des éditions

Au montage des composants `Registration.jsx`, `Tickets.jsx` et `ScheduleSection.jsx`, le hook personnalisé `useEditions` interroge la table `festival_editions` de Supabase pour récupérer la liste des éditions actives. Les éditions ne sont plus définies en dur dans le code source : toute modification effectuée dans l'`EditionsManager` du back-office est immédiatement reflétée dans l'interface publique.

### 5.3 Parcours 2 — Inscription au concours

Le visiteur saisit son e-mail (délai de 500 ms) — le composant `Registration.jsx` interroge la table `registrations` pour détecter les éditions déjà inscrites et les affiche en temps réel. À la soumission, les données sont insérées dans `registrations` (RLS anon INSERT autorisé), puis l'Edge Function `send-confirmation` génère et envoie un e-mail HTML personnalisé.

### 5.4 Parcours 3 — Réservation de billets

À la soumission : (1) insertion dans `ticket_orders` (RLS anon INSERT) ; (2) appel à l'Edge Function `send-ticket` qui génère N numéros au format `SWF-{EDI}-{6 chars}`, insère les billets dans `tickets` (`service_role`), génère les PDF via `pdf-lib` avec QR codes et envoie les billets en pièces jointes via SMTP ; (3) si `newsletter_opt_in = true`, upsert dans `newsletter_subscribers`.

### 5.5 Parcours 4 — Formulaire de contact

La soumission de la modal `ContactModal.jsx` invoque l'Edge Function `send-contact`, qui transmet le message par e-mail à `info@snow-wonder.be`. Aucune insertion en base de données n'est effectuée.

### 5.6 Parcours 5 — Galerie photo (lecture publique)

Au montage de `GallerySection.jsx`, une requête `SELECT` filtre les images actives (`active = true`, `section = 'gallery'`) ordonnées par position. Un lightbox implémenté manuellement gère la navigation clavier.

### 5.7 Parcours 6 — Scan à l'entrée (TicketScanner)

L'agent de contrôle accède à l'interface via le rôle `scanner`. Le composant `TicketScanner.jsx` utilise `html5-qrcode` pour accéder à la caméra et décoder les QR

codes en temps réel. L'interface retourne un code couleur immédiat : vert (validation réussie), orange (déjà scanné), rouge (introuvable).

### 5.8 Parcours 7 — Authentification Back-Office

La connexion passe par `supabase.auth.signInWithPassword()`, suivie d'une double vérification dans `admin_users`. Le rôle détermine l'interface affichée : panneau complet pour `admin/super_admin`, interface de scan pour `scanner`.

### 5.9 Parcours 8 — Back-Office (opérations CRUD admin)

| Onglet       | Opérations                                                                                |
|--------------|-------------------------------------------------------------------------------------------|
| Dashboard    | SELECT COUNT registrations, ticket_orders — actions rapides — journal d'activité récente  |
| Éditions     | SELECT / INSERT / UPDATE / DELETE FROM festival_editions — aperçu icône Material Symbols  |
| Inscriptions | SELECT / UPDATE / DELETE FROM registrations                                               |
| Billetterie  | SELECT / UPDATE / DELETE FROM ticket_orders, tickets                                      |
| Galerie      | SELECT / INSERT / UPDATE / DELETE FROM carousel_images                                    |
| Gagnants     | SELECT / INSERT / UPDATE / DELETE FROM winners                                            |
| Scanner      | Lecture/mise à jour de tickets (used/used_at) et registrations (checked_in/checked_in_at) |

### 5.10 Tableau récapitulatif des permissions RLS

| TABLE                  | INSERT        | SELECT                  | UPDATE / DELETE         |
|------------------------|---------------|-------------------------|-------------------------|
| registrations          | anon          | authenticated           | authenticated           |
| ticket_orders          | anon          | authenticated           | authenticated           |
| tickets                | service_role  | authenticated + scanner | authenticated + scanner |
| festival_editions      | authenticated | anon + auth             | authenticated           |
| newsletter_subscribers | anon          | authenticated           | authenticated           |
| carousel_images        | authenticated | anon + auth             | authenticated           |
| winners                | authenticated | anon + auth             | authenticated           |
| rate_limits            | service_role  | service_role            | service_role            |
| admin_users            | —             | authenticated           | —                       |

### 5.11 Points clés de sécurité

Trois principes structurent la sécurité des flux de données de l'application.

Jamais de secret côté client : la clé anonyme Supabase (ANON\_KEY) est publique par conception et ne permet que les opérations autorisées par les politiques RLS. Les secrets SMTP et la clé service\_role ne vivent que dans les variables d'environnement des Edge Functions, côté serveur.

RLS comme unique garde-fou : c'est la seule couche qui empêche un visiteur de lire les données des autres utilisateurs. Elle est configurée directement au niveau PostgreSQL, indépendamment du code applicatif.

Flux splitté pour la billetterie : le client insère la commande directement (INSERT dans ticket\_orders), puis délègue la logique sensible à l'Edge Function send-ticket (service\_role) pour insérer les billets, générer les PDF et envoyer les e-mails (voir Figure 7).

```

Deno.serve(async (req) => {
  if (req.method === "OPTIONS") {
    return new Response("ok", { headers: corsHeaders });
  }

  try {
    const {
      name: nom,
      email,
      edition,
      editionLabel = edition,
      editionDate = "",
      editionTime = "",
      quantity: quantite,
      orderId: idCommande,
    } = await req.json();
    console.log(
      `send-confirmation-billets: ${email} edition=${edition} qte=${quantite}`,
    );

    // 5 envois max par email par heure (protection anti-spam)
    const allowed = await checkRateLimit(
      `ticket:${email.toLowerCase()}`,
      5,
      3600,
    );
    if (!allowed) {
      return new Response(JSON.stringify({ error: "Trop de requêtes." }), {
        status: 429,
        headers: { ...corsHeaders, "Content-Type": "application/json" },
      );
    }

    const billets = Array.from({ length: quantite }, () => ({
      ticket_number: genererNumeroBillet(edition),
      order_id: idCommande,
      edition,
      holder_name: nom,
      holder_email: email,
    }));

    const dbRes = await fetch(`${SUPABASE_URL}/rest/v1/tickets`, {
      method: "POST",
      headers: {
        Authorization: `Bearer ${SUPABASE_SERVICE_KEY}`,
        apikey: SUPABASE_SERVICE_KEY,
        "Content-Type": "application/json",
        Prefer: "return=minimal",
      },
      body: JSON.stringify(billets),
    });

    if (!dbRes.ok) {
      const err = await dbRes.text();
      throw new Error(`Échec insertion DB : ${err}`);
    }

    let piecesJointes: object[] = [];
    try {
      piecesJointes = await Promise.all(
        billets.map(async (b) => {
          const pdfBytes = await genererBilletPDF(
            b.ticket_number,
            nom,
            edition,
            idCommande,
            editionLabel,
            editionDate,
            editionTime,
          );
          return {
            filename: `billet-${b.ticket_number}.pdf`,
            content: Buffer.from(pdfBytes),
            contentType: "application/pdf",
          };
        })
      );
    } catch (erreurPdf) {
      console.log(`${piecesJointes.length} PDF(s) prêt(s)`);
      console.error("Échec PDF :", erreurPdf);
    }

    const html = emailConfirmationBillets({
      nom,
      email,
      edition,
      editionLabel,
      editionDate,
      editionTime,
      quantite,
      idCommande,
    });

    const transporteur = nodemailer.createTransport({
      host: SMTP_HOST,
      port: SMTP_PORT,
      secure: SMTP_PORT === 465,
      auth: { user: SMTP_USER, pass: SMTP_PASS },
    });

    await transporteur.sendMail({
      from: FROM,
      to: email,
      subject: `Vos billets - Snow Wonder Festival ${edition}`,
      html,
      attachments: piecesJointes,
    });

    console.log("Email envoyé");
  }
});

```

Figure 7 — Extrait de code — Edge Function send-ticket (génération PDF + envoi e-mail)

## Chapitre 6 — Fonctionnalités clés du site

### 6.1 Le compte à rebours dynamique (Hero)

La section Hero affiche un compte à rebours en temps réel jusqu'à la prochaine édition du festival. La logique repose sur un hook React personnalisé `useCountdown` qui calcule la différence entre la date cible et l'instant présent toutes les secondes via `setInterval`. Les éditions sont chargées dynamiquement via `useEditions`, et la fonction `getNextEdition` retourne automatiquement la prochaine date future.

Des animations `animate-ping` (effets de scintillement d'étoiles) sont positionnées autour du logo pour créer un effet immersif et hivernal. Sur mobile, un clic sur le logo flocon de neige déclenche une animation de chute de neige interactive : 15 flocons sont générés avec des paramètres aléatoires (position, taille, durée, délai) et tombent en fondu depuis le haut de l'écran. Positionnés en `fixed` avec `pointerEvents: none`, ils ne perturbent aucune interaction et disparaissent automatiquement après quelques secondes.

### 6.2 Gestion dynamique des éditions

Le module `EditionsManager` (`src/admin/EditionsManager.jsx`) permet à l'administrateur de créer, modifier, activer/désactiver et supprimer des éditions sans aucun redéploiement. Les composants `Registration.jsx`, `Tickets.jsx` et `ScheduleSection.jsx` consomment les éditions via le hook `useEditions`, reflétant en temps réel toute modification effectuée dans l'administration (voir Figure 8).

**Snow Wonder** ADMIN-IN-THE-SNOW ADMIN DÉCONNEXION

TABLEAU DE BORD INSCRIPTIONS BILLETTERIE SCANNER ÉDITIONS GALERIE GAGNANTS

### Éditions du festival

Gérez les éditions affichées sur le site et dans les formulaires [+ Nouvelle édition](#)

| Édition              | Date                                 | Statut  | Actions                         |
|----------------------|--------------------------------------|---------|---------------------------------|
| Édition d'Ouverture  | 6 décembre 2026 · Édition Ouverture  | Actif   | Désactiver, Modifier, Supprimer |
| Édition Mi-Hiver     | 10 janvier 2027 · Édition Mi-Hiver   | Actif   | Désactiver, Modifier, Supprimer |
| Grande Finale        | 7 février 2027 · Grande Finale       | Actif   | Désactiver, Modifier, Supprimer |
| test                 | 31 main 2026 · Édition Spéciale      | Inactif | Activer, Modifier, Supprimer    |
| Quam qui deleniti qu | In occaecat quis mod · Grande Finale | Inactif | Activer, Modifier, Supprimer    |

#### Nouvelle édition

IDENTIFIANT \* LABEL \* MOIS

ex: december\_2026 Édition Décembre Décembre

Clé unique, non modifiable après création

DATE AFFICHÉE DATE ISO \* ORDRE

6 décembre 2026 jj/mm/aaaa --:-- 0

THÈME APERÇU

— Choisir un thème — #4289b6 - ac\_unit

DESCRIPTION

ÉVÉNEMENTS DU PROGRAMME

Trophée amateur Description de l'événement

+ Ajouter un événement

☒ Édition active (visible sur le site et dans les formulaires)

Annuler Enregistrer

Figure 8 — Onglet Éditions du back-office — formulaire de création/modification d'une édition

### 6.3 Les formulaires : inscription et billetterie

La section formulaires est l'une des parties les plus riches fonctionnellement du site. Elle comprend deux formulaires distincts présentés côte à côte dans une mise en scène visuelle élaborée. Les deux utilisent le hook `useEditions` pour charger dynamiquement la liste des éditions depuis Supabase.

Formulaire d'inscription au concours (`Registration.jsx`) : ce formulaire permet aux participants de s'inscrire en choisissant leur catégorie (amateur ou professionnel) et leur(s) édition(s). Un mécanisme de détection en temps réel vérifie, après saisie de l'e-mail, si l'utilisateur est déjà inscrit à certaines éditions — celles-ci sont alors

désactivées dans la liste. À la soumission, seules les éditions non encore inscrites sont insérées, et une Edge Function envoie un e-mail de confirmation par édition.

Formulaire de billetterie (Tickets.jsx) : ce formulaire permet à tout visiteur de réserver des billets gratuits, dans la limite de 10 billets par adresse e-mail et par édition. Le champ « demandes spéciales » est limité à 1 000 caractères. Chaque billet PDF est généré côté serveur avec pdf-lib et inclut un QR code unique, le nom du porteur et la référence de commande (voir Figure 9).

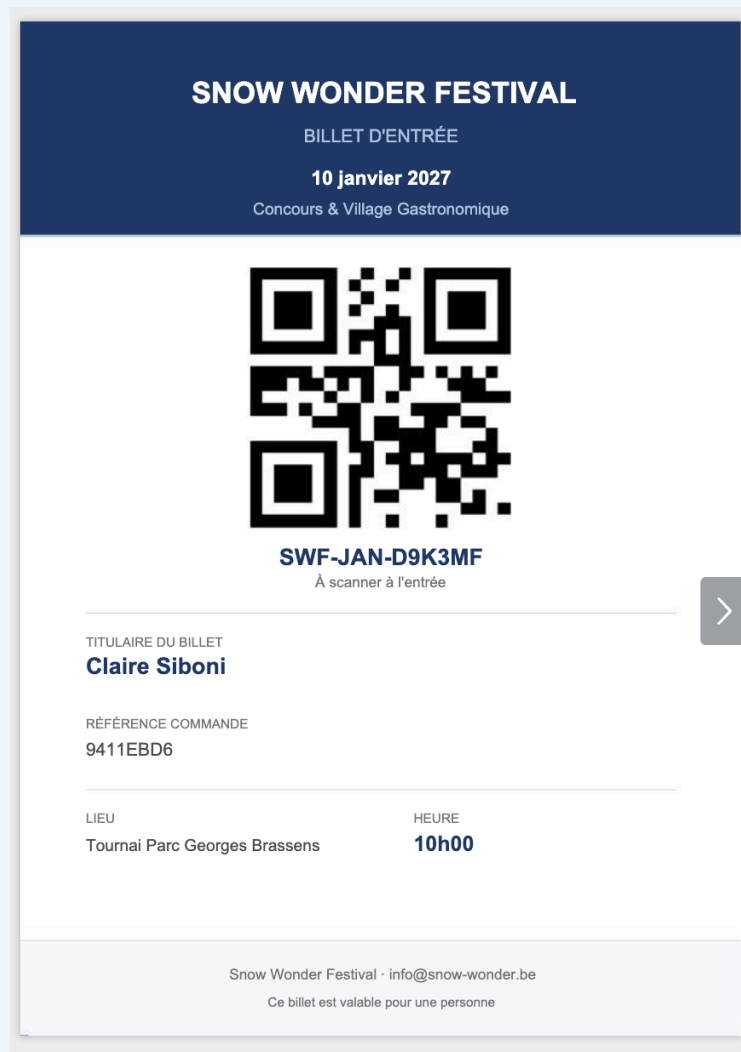


Figure 9 — Exemple de billet PDF généré avec QR code

## 6.4 La galerie photo dynamique

La galerie (GallerySection.jsx) charge ses images directement depuis carousel\_images de Supabase, en filtrant les images actives de la section gallery, ordonnées par position. Un lightbox a été implémenté manuellement, avec navigation clavier (flèches, Escape) et compteur de position.



## 6.5 Le système de scan à l'entrée (TicketScanner)

Un système complet de validation à l'entrée a été développé autour de TicketScanner.jsx, s'appuyant sur html5-qrcode pour accéder à la caméra et décoder les QR codes en temps réel. L'interface affiche un retour visuel immédiat : vert (validé), ambre (déjà scanné), rouge (introuvable). Une saisie manuelle reste disponible en alternative. Un nouveau rôle scanner dans admin\_users donne accès à cette interface épurée sans accès au back-office (voir Figure 10).

[ Insérer capture d'écran ici ]

Figure 10 — Interface TicketScanner — retour visuel vert / ambre / rouge

## 6.6 Le panneau d'administration (Back-Office)

L'interface d'administration est accessible via l'URL /#admin. L'authentification utilise le système JWT de Supabase avec double vérification (Supabase Auth + table admin\_users). Le rôle détermine l'interface affichée : panneau complet (7 onglets) pour admin/super\_admin, interface de scan pour scanner.

Le panneau est structuré en sept onglets : Tableau de bord (statistiques, actions rapides, journal d'activité), Inscriptions (filtrage et suppression), Billetterie (commandes et billets individuels), Scanner (lecteur QR avec retour visuel vert/ambre/rouge), Éditions (gestion complète sans redéploiement), Galerie (ajout, réorganisation, activation) et Gagnants (palmarès administrable). Voir Figures 11 et 12.

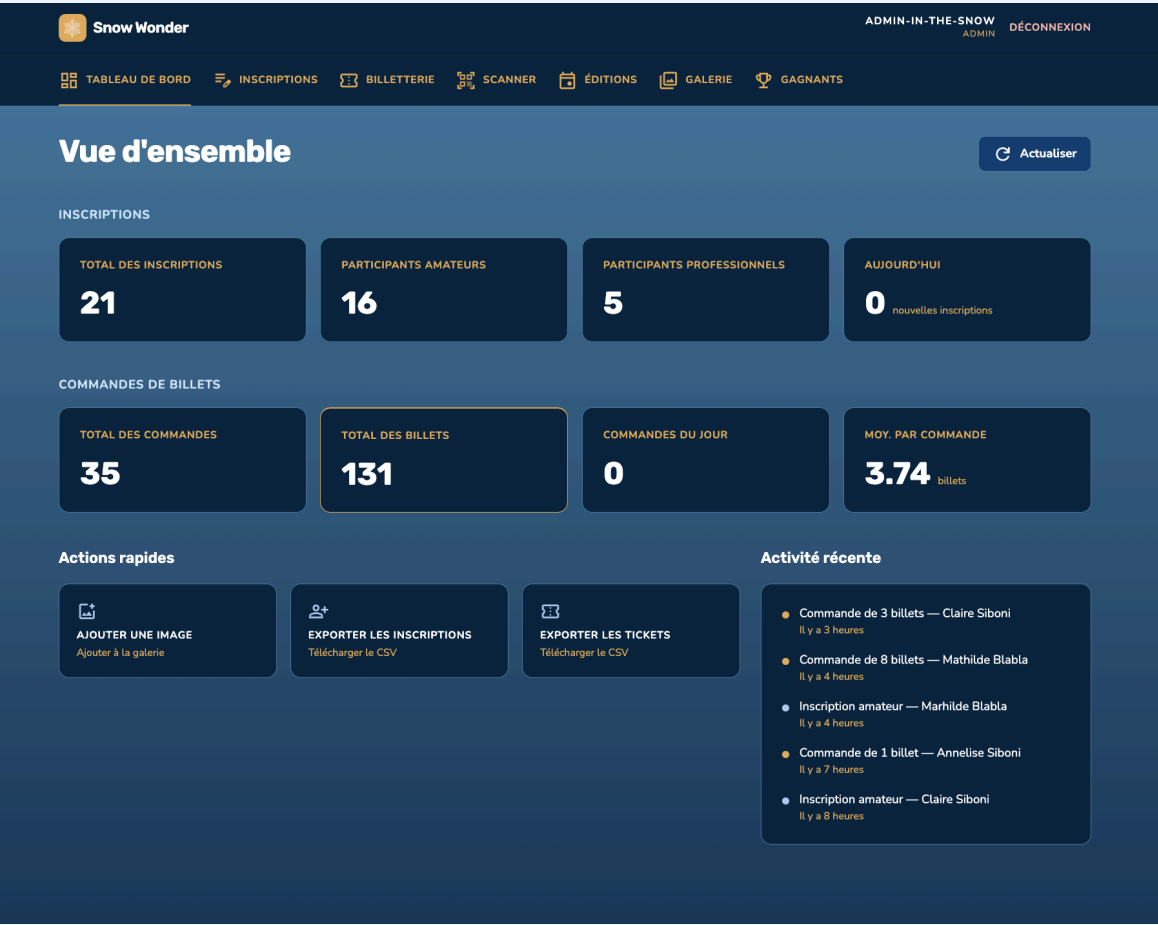


Figure 11 — Tableau de bord du back-office — statistiques et journal d'activité

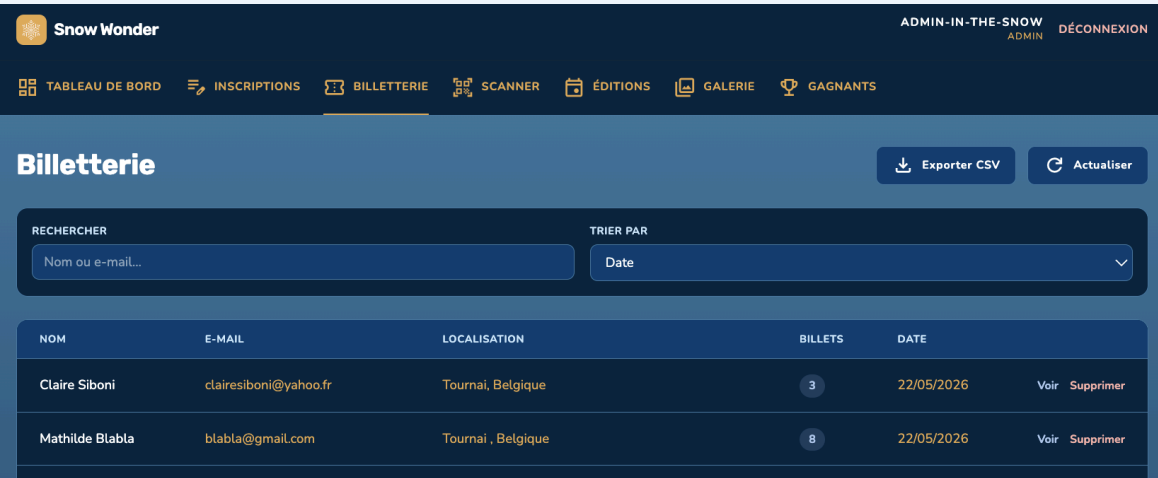


Figure 12 — Onglet Billetterie — liste des commandes et billets générés

## 6.7 Sécurité et protection des données

La sécurité repose sur quatre couches complémentaires : Row Level Security (RLS) sur toutes les tables Supabase, variables d'environnement (secrets SMTP et clé Mapbox jamais exposés côté client), validation et déduplication des données (côté client et contraintes SQL), et rate limiting sur les trois Edge Functions d'envoi d'e-mails.

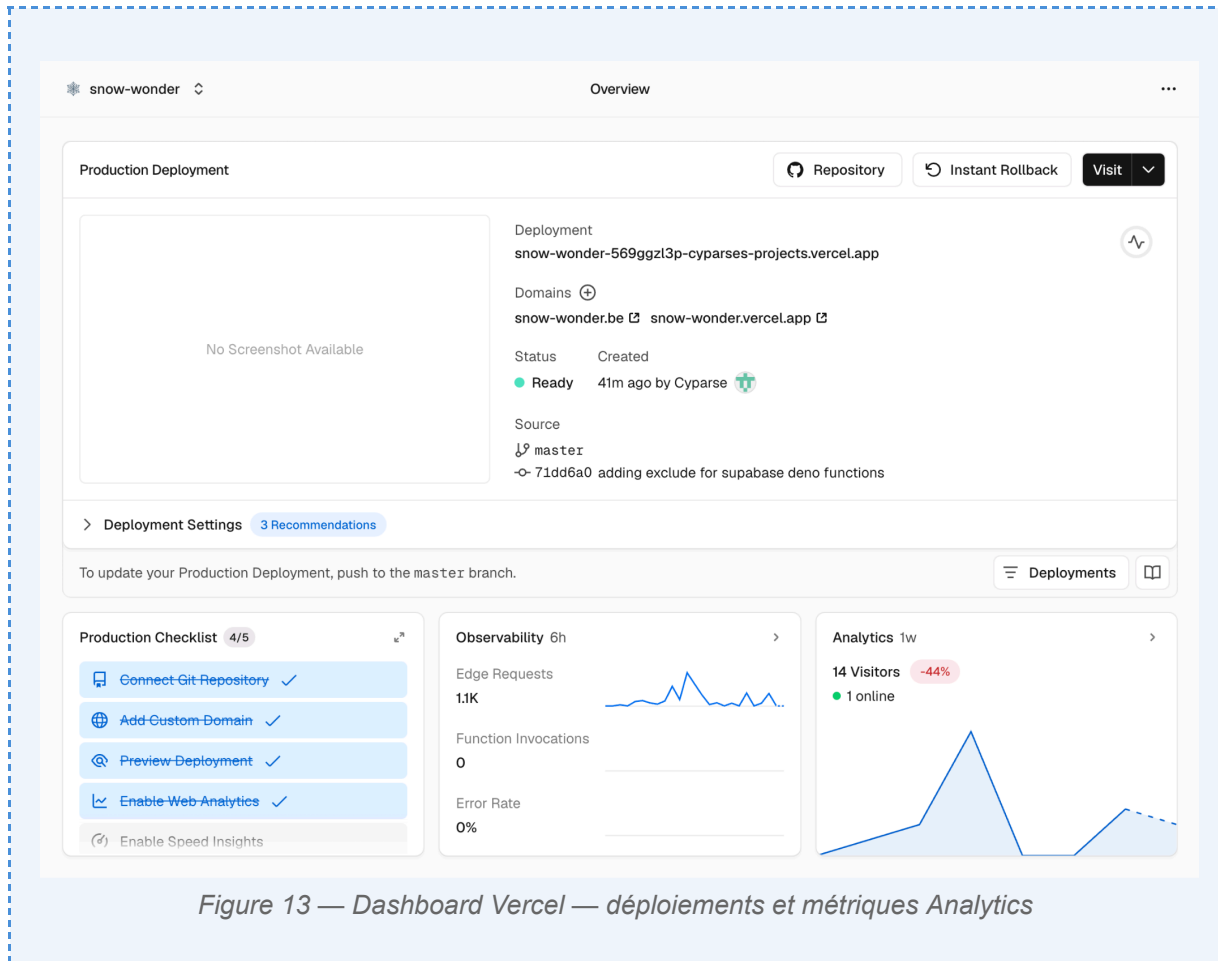
| Fonction                                 | Limite                          |
|------------------------------------------|---------------------------------|
| send-confirmation (inscription concours) | 5 envois par e-mail par heure   |
| send-ticket (billetterie)                | 5 envois par e-mail par heure   |
| send-contact (formulaire de contact)     | 3 messages par e-mail par heure |

Lorsque la limite est atteinte, la fonction retourne un code HTTP 429 Too Many Requests. La fonction send-contact applique en outre un échappement HTML explicite de toutes les variables avant insertion dans le template. React échappe automatiquement les valeurs interpolées dans le JSX (protection XSS).

## Chapitre 7 — Déploiement et mise en production

### 7.1 Déploiement sur Vercel

Le site est déployé sur Vercel, qui détecte automatiquement le projet Vite, lance npm run build et sert le dossier dist/ généré. Vercel Analytics a été intégré pour suivre les métriques de trafic de manière respectueuse de la vie privée (voir Figure 13).



### 7.2 Supabase Edge Functions

Les Edge Functions Deno (send-confirmation, send-contact, send-ticket) sont déployées sur l'infrastructure Supabase, exécutées à la périphérie du réseau. La gestion des en-têtes CORS est mutualisée dans un fichier partagé `_shared/cors.ts`.

### 7.3 Optimisation des performances

Un audit Lighthouse réalisé sur le site déployé a mis en évidence quatre catégories de problèmes : un bundle JavaScript monolithique de 296 Ko, des images manquant d'attributs width/height générateurs de CLS, des ressources graphiques trop lourdes et six tâches longues sur le thread principal.

#### Découpage du bundle JavaScript (code splitting)

Grâce à React.lazy et <Suspense>, les sections sous la ligne de flottaison et la totalité du back-office sont chargées à la demande. La configuration de Vite définit des chunks stables : vendor-react, vendor-supabase, vendor-leaflet et admin. Le chunk initial index.js est descendu de 296 Ko à environ 220 Ko.

#### **Lazy loading et priorisation des images**

L'attribut loading=« lazy » et decoding=« async » a été appliqué à toutes les images hors de la zone visible initiale. L'image LCP (HeroImage) reçoit fetchpriority=« high » pour un téléchargement prioritaire.

#### **Conversion WebP et réduction du poids des images**

Les trois images locales les plus lourdes ont été converties au format WebP avec recompression à 80 % : fond du marché de 8,1 Mo à 231 Ko, footer de 2,4 Mo à 111 Ko, image héro de 1,4 Mo à 388 Ko. Les images de la galerie sur Supabase Storage devront être ré-uploadées en WebP compressé (1 200 px maximum).

## Chapitre 8 — RGPD, cookies et pollution numérique

### 8.1 RGPD et protection des données personnelles

Le Règlement Général sur la Protection des Données (RGPD) encadre la collecte, le stockage et le traitement de toute donnée personnelle au sein de l'Union européenne. Dans le cadre de Snow Wonder Festival, deux catégories de données personnelles sont collectées selon le formulaire utilisé.

Pour l'inscription au concours : prénom, nom, adresse e-mail, numéro de téléphone et organisation éventuelle.

Pour la réservation de billets : prénom, nom, adresse e-mail, numéro de téléphone et adresse postale complète (rue, ville, code postal, pays).

Les mesures de conformité mises en œuvre sont les suivantes. Les formulaires soumettent les données exclusivement via HTTPS chiffré. Les politiques RLS de PostgreSQL garantissent que les visiteurs anonymes ne peuvent qu'insérer des données. Les secrets sont stockés dans les variables d'environnement Supabase. Les utilisateurs acceptent explicitement les conditions via une case à cocher obligatoire avant toute soumission, conformément au principe de consentement éclairé du RGPD. L'administrateur peut supprimer toute inscription ou commande pour honorer les demandes de droit à l'effacement.

### 8.2 Gestion des cookies

L'application utilise des cookies techniques Supabase pour la gestion des sessions JWT de l'administrateur, strictement nécessaires au fonctionnement du back-office. Vercel Analytics collecte des métriques agrégées et anonymisées, sans cookies de traçage tiers.

### 8.3 Pollution numérique

Le numérique représente environ 4 % des émissions mondiales de gaz à effet de serre selon l'ADEME. Snow Wonder Festival a été conçu avec quelques principes réduisant son empreinte : architecture SPA limitée en requêtes réseau, tree-shaking et code-splitting via Vite, gestion dynamique des images depuis Supabase Storage, et Edge Functions exécutées à la périphérie. Des améliorations futures pourraient inclure un CDN dédié avec compression WebP automatique.

## Chapitre 9 — Estimation budgétaire

### 9.1 Hypothèses de calcul

L'estimation repose sur le tarif horaire moyen d'un développeur web junior en Belgique, soit 29,24 €/heure brut selon Joooble<sup>1</sup>. Le projet étant réalisé en totale autonomie, aucun coût de coordination d'équipe n'est à prévoir.

### 9.2 Phases de développement

| Phase                   | Description                                               | Durée | Coût       |
|-------------------------|-----------------------------------------------------------|-------|------------|
| Analyse                 | Brief client, impact mapping, user stories, planification | 10h   | 292,40 €   |
| Conception              | Charte graphique, wireframes, schéma de base de données   | 20h   | 584,80 €   |
| Développement front-end | Composants React, TailwindCSS, intégration Leaflet/Mapbox | 60h   | 1 754,40 € |
| Développement back-end  | Configuration Supabase, RLS, Edge Functions, back-office  | 50h   | 1 462,00 € |
| Tests et débogage       | Tests fonctionnels multi-navigateurs, corrections         | 15h   | 438,60 €   |
| Déploiement             | Mise en ligne Vercel, migration base de données, domaine  | 10h   | 292,40 €   |
| Total développement     |                                                           | 165h  | 4 824,60 € |

### 9.3 Frais annexes (première année)

| Poste                                                                | Coût annuel estimé |
|----------------------------------------------------------------------|--------------------|
| Nom de domaine snow-wonder.be (OVH)                                  | ~10,14 €           |
| Adresse e-mail professionnelle (OVH, 0,30 €/mois)                    | ~3,60 €            |
| Hébergement Vercel (plan gratuit)                                    | 0 €                |
| Supabase (plan gratuit, jusqu'à 50 000 utilisateurs actifs mensuels) | 0 €                |
| Total frais techniques                                               | ~13,74 €           |

### 9.4 Maintenance annuelle

La maintenance couvre les mises à jour des dépendances (React, TailwindCSS v4, Supabase SDK), la surveillance des performances, les corrections de bugs éventuels

<sup>1</sup>JOOBLE, joooble.org, <https://be.joooble.org/salary/juniordeveloper>, 21/05/2026, 11/12/2024 — « La statistique du tarif horaire moyen d'un développeur junior en Belgique m'a permis d'établir les critères de rémunération »

et l'ajout de contenu dans le back-office. Elle est estimée à 10 heures par an, soit 292,40 €.

### 9.5 Coût total première année

| Poste            | Montant     |
|------------------|-------------|
| Développement    | 4 824,60 €  |
| Frais techniques | ~13,74 €    |
| Maintenance      | 292,40 €    |
| Total            | ~5 130,74 € |

Ce budget constitue une estimation indicative pour un développeur junior travaillant en autonomie.



## Conclusion

### Intégration des savoirs et savoir-faire

Ce projet a été l'occasion de mettre en pratique, souvent pour la première fois en conditions réelles, des compétences que la formation avait introduites sans toujours pouvoir les approfondir. Configurer des politiques RLS directement au niveau PostgreSQL, orchestrer des Edge Functions Deno pour l'envoi d'e-mails transactionnels et la génération de PDF, ou encore intégrer proprement une bibliothèque DOM impérative comme Leaflet dans un environnement React — chacun de ces défis m'a forcée à aller chercher une compréhension plus profonde des mécanismes sous-jacents, plutôt que de me contenter d'une solution qui fonctionne.

Le volet design a été tout aussi formateur. Concevoir trois expériences visuelles distinctes pour mobile, tablette et desktop — sans simplement masquer des éléments, mais en repensant réellement la mise en page à chaque breakpoint — m'a appris à raisonner en termes de contexte d'utilisation autant qu'en termes de technique.

### Adéquation entre objectifs et résultats

Les objectifs définis dans le cahier des charges ont été intégralement atteints et dépassés :

- Le site est une single-page application complète avec front-office et back-office ;
- La carte interactive Leaflet/Mapbox est fonctionnelle avec marqueurs personnalisés ;
- Le système de réservation génère des billets PDF individuels envoyés par e-mail, chacun doté d'un QR code unique ;
- Le back-office permet une gestion autonome du contenu, y compris la création et la modification des éditions sans redéploiement ;
- Un système de contrôle des entrées par QR code a été développé et intégré, avec un rôle dédié dans la gestion des accès.

### Difficultés rencontrées et solutions apportées

La refonte technologique en cours de projet — passage du HTML/CSS/JS vanilla vers React et TailwindCSS — a constitué le premier défi majeur. Ce choix, motivé par mon expérience en stage et par la nature SPA du projet, a impliqué de reprendre une base de code existante et de la restructurer entièrement, tout en continuant à avancer sur les fonctionnalités.

La mise en page de la section formulaires sur desktop a nécessité de nombreuses itérations pour trouver l'équilibre entre l'effet visuel des deux cercles translucides superposés et l'utilisabilité réelle des formulaires qu'ils contiennent.

La gestion dynamique des éditions a quant à elle imposé une refonte de plusieurs composants qui reposaient initialement sur un dictionnaire statique, ainsi que la correction d'un bug RLS empêchant l'inscription à la newsletter lors de la création d'une nouvelle édition.

## Réflexion personnelle

Snow Wonder Festival est le projet qui m'a le plus appris — non pas parce qu'il était le plus guidé, mais parce qu'il ne l'était pas du tout. Prendre des décisions d'architecture, assumer un changement technologique en cours de route, déboguer seule des comportements inattendus en production : tout cela m'a donné une vision bien plus concrète de ce que représente le développement web au quotidien. Je repars de ce TFE avec la conviction que c'est dans cet environnement, exigeant, créatif et en constante évolution, que je veux continuer à progresser.

## Bibliographie

### Ouvrages et règlements

- COMMISSION EUROPÉENNE, europa.eu, Règlement (UE) 2016/679 — RGPD, <https://eur-lex.europa.eu/legal-content/FR/TXT/?uri=CELEX%3A32016R0679>, 21/05/2026, 27/04/2016
- ADEME, ademe.fr, La face cachée du numérique, <https://www.ademe.fr/la-face-cachee-du-numerique>, 21/05/2026, 2019

### Documentations techniques

- REACT, react.dev, Documentation officielle React 19, <https://react.dev>, 21/05/2026
- VITEJS, vitejs.dev, Documentation officielle Vite, <https://vitejs.dev>, 21/05/2026
- TAILWINDCSS, tailwindcss.com, Documentation officielle TailwindCSS v4, <https://tailwindcss.com>, 21/05/2026
- SUPABASE, supabase.com, Documentation officielle — Row Level Security, <https://supabase.com/docs/guides/auth/row-level-security>, 21/05/2026
- LEAFLET, leafletjs.com, Documentation officielle Leaflet.js, <https://leafletjs.com>, 21/05/2026
- MAPBOX, mapbox.com, Documentation Mapbox Styles API, <https://docs.mapbox.com/api/maps/styles/>, 21/05/2026
- VERCEL, vercel.com, Documentation officielle Vercel, <https://vercel.com/docs>, 21/05/2026
- HTML5-QRCODE, github.com/mebjas/html5-qrcode, Documentation officielle html5-qrcode, <https://github.com/mebjas/html5-qrcode>, 21/05/2026

### Sources économiques

- JOOBLE, jooble.org, Salaire développeur junior Belgique, <https://be.jooble.org/salary/juniordeveloper>, 21/05/2026, 11/12/2024

### Crédits photographiques

- NESTER Siarhei, pexels.com, Photo neige/hiver, <https://www.pexels.com/fr-fr/photo/neige-eau-hiver-glace-18858820/>, 21/05/2026
- SIDDIQUE Saif, pexels.com, Sculpture de neige grenouille guitariste, <https://www.pexels.com/fr-fr/photo/sculpture-de-neige-d-une-grenouille-jouant-de-la-guitare-a-frankenmuth-35252683/>, 21/05/2026
- GROSSGASTEIGER Eberhard, pexels.com, Lampe château hivernal, <https://www.pexels.com/fr-fr/photo/lampe-a-theme-chateau-beige-776389/>, 21/05/2026
- RADIK 2707, pexels.com, Sculpture de glace nocturne, <https://www.pexels.com/fr-fr/photo/creation-d-une-sculpture-de-glace-nocturne-en-noir-et-blanc-30238611/>, 21/05/2026

- SOMMACAL Francesco, pexels.com, Enfant et bonhomme de neige, <https://www.pexels.com/fr-fr/photo/enfant-construisant-un-grand-bonhomme-de-neige-dans-un-paysage-hivernal-36242337/>, 21/05/2026
- BOLTNEVA Valeria, pexels.com, Hiver froid, <https://www.pexels.com/fr-fr/photo/rhume-froid-neige-hiver-14817812/>, 21/05/2026
- S. Natalia, pexels.com, Famille bonhommes de neige, <https://www.pexels.com/fr-fr/photo/charmante-famille-de-bonhommes-de-neige-lors-d-une-journee-d-hiver-30129593/>, 21/05/2026
- YUAN Yuan, pexels.com, Bonhomme de neige décoré, <https://www.pexels.com/fr-fr/photo/joyeux-bonhomme-de-neige-decore-de-feuilles-d-automne-29567493/>, 21/05/2026
- AUTEUR INCONNU, pxhere.com, Photo hivernale, <https://pxhere.com/en/photo/1072102>, 21/05/2026
- AUTEUR INCONNU, pxhere.com, Photo hivernale, <https://pxhere.com/en/photo/1184648>, 21/05/2026
- AUTEUR INCONNU, wikimedia.org, Harbin Ice Festival sculpture, [https://upload.wikimedia.org/wikipedia/commons/a/a4/Moomen\\_in\\_Snew%2C\\_Harbin\\_International\\_Ice\\_and\\_Snow\\_Sculpture\\_Festival\\_%283238489838%29.jpg](https://upload.wikimedia.org/wikipedia/commons/a/a4/Moomen_in_Snew%2C_Harbin_International_Ice_and_Snow_Sculpture_Festival_%283238489838%29.jpg), 21/05/2026
- HARKNESS Derek, flickr.com, Harbin festival photo, <https://www.flickr.com/photos/derekharkness/3237690883>, 21/05/2026
- AUTEUR INCONNU, wikimedia.org, Snow steam train Harbin, [https://upload.wikimedia.org/wikipedia/commons/9/94/Snow\\_steam\\_train%2C\\_Harbin\\_International\\_Ice\\_and\\_Snow\\_Sculpture\\_Festival\\_%283237649999%29.jpg](https://upload.wikimedia.org/wikipedia/commons/9/94/Snow_steam_train%2C_Harbin_International_Ice_and_Snow_Sculpture_Festival_%283237649999%29.jpg), 21/05/2026

### Intelligence artificielle

Claude (Anthropic), claude.ai, base de connaissances mise à jour août 2025, consulté en mai 2026. Utilisé pour reformuler et améliorer la rédaction de certains passages du TFE<sup>2</sup>.

---

<sup>2</sup>Claude (Anthropic), claude.ai, base de connaissances mise à jour août 2025, consulté en mai 2026. Utilisé pour reformuler et améliorer la rédaction de certains passages du TFE.

## Table des figures

- Figure 1 — Page d'accueil — section Hero avec compte à rebours (desktop)
- Figure 2 — Section formulaires — mise en page desktop (cercles superposés)
- Figure 3 — Section formulaires — mise en page mobile (quart de cercle)
- Figure 4 — Carte principale Leaflet — marqueur du site du festival
- Figure 5 — Carte du marché — modal avec les 9 marqueurs gastronomiques
- Figure 6 — Vue des tables dans l'interface Supabase (Table Editor ou diagramme ERD)
- Figure 7 — Extrait de code — Edge Function send-ticket (génération PDF + envoi e-mail)
- Figure 8 — Onglet Éditions du back-office — formulaire de création/modification d'une édition
- Figure 9 — Exemple de billet PDF généré avec QR code
- Figure 10 — Interface TicketScanner — retour visuel vert / ambre / rouge
- Figure 11 — Tableau de bord du back-office — statistiques et journal d'activité
- Figure 12 — Onglet Billetterie — liste des commandes et billets générés
- Figure 13 — Dashboard Vercel — déploiements et métriques Analytics